IV Studio - PRODUCT BACKLOG

Project: Uni-Helper
Integrants: Andy Tiban, Nohemi Tocagon, Tomas Tapia, Danilo Simba

1. Adaptive Schedule Manager

TASKS:

- Record schedule entries using standardized block and room identifiers (e.g., blocks A through H).
- Validate current or future days for scheduling.
- Generate a structured weekly and daily agenda view.
  Non-functional requirements:
- Reject invalid room codes instantly using regex validation to ensure they strictly fall within allowed blocks (A-H, excluding restricted ones).
- Local data serialization via Gson must load the schedule object into memory in under 500ms.
  STARTED TASKS: Yes
  DONE: Not yet. The core logic works, but it still needs polishing for a complete and stable release.

2. Smart Budgeting Modes

TASKS:

- Offer customizable spending modes (`Save`, `Recommended`, `Social`).
- Calculate a personalized daily budget based on campus dining, commute choice, and academic surcharges.
  Non-functional requirements:
- All financial calculations must use `BigDecimal` or precise integer arithmetic (cents) to avoid floating-point rounding errors native to Java.
- Financial data inputs must catch `InputMismatchException` properly in the console.
  STARTED TASKS: Yes
  DONE: Not yet. The core logic works, but it still needs polishing for a complete and stable release.

3. Task Priority System

TASKS:

- Allow users to assign High, Medium, and Low priority to tasks.
- Automatically sort and categorize academic assignments based on urgency and priority tags.
  Non-functional requirements:

- The priority options must be strongly typed using Java `Enum` (HIGH, MEDIUM, LOW).
- The system must sort task lists efficiently using Java's native `Collections.sort()` or `Stream` API without noticeably freezing the CLI.
  STARTED TASKS: Yes
  DONE: Not yet. The core logic works, but it still needs polishing for a complete and stable release.

4. Quick Study Break Reminder

TASKS:

- Provide automated reminders based on cumulative study time.
- Suggest short, healthy breaks (active pauses) to prevent cognitive fatigue.
  Non-functional requirements:
- The break timer must utilize lightweight background threads (e.g., `java.util.Timer` or `ScheduledExecutorService`) so it does not block the main CLI thread.
  STARTED TASKS: Yes
  DONE: Not yet. The core logic works, but it still needs polishing for a complete and stable release.

5. Subject Filter

TASKS:

- Allow students to filter their task lists and views by specific subjects.
- Display only tasks and records associated with the currently selected course.
  Non-functional requirements:
- The search and filter operations must be case-insensitive (`String.equalsIgnoreCase()`) to improve user experience.
- Filtering must happen on the in-memory lists to ensure instant console output response times.
  STARTED TASKS: Yes
  DONE: Not yet. The core logic works, but it still needs polishing for a complete and stable release.

6. Task Registration and Management

TASKS:

- Create a prompt-based task registration flow.
- Add options to edit and delete existing tasks, including deadlines.
  Non-functional requirements:
- Console inputs using `java.util.Scanner` must be properly sanitized to prevent buffer issues (like the newline character skipping bug).

- Tasks must be tightly encapsulated in OOP classes to prevent unauthorized modifications to attributes.
  STARTED TASKS: Yes
  DONE: Not yet. The core logic works, but it still needs polishing for a complete and stable release.

7. User Login and Authentication

TASKS:

- Create the login and registration flow (username and password).
- Verify credentials against saved data to manage secure student access.
  Non-functional requirements:
- Passwords must NEVER be stored in plain text; they must be hashed using SHA-256 (`java.security.MessageDigest`) before saving to `users.json`.
- Session persistence must reliably link the active user to their specific JSON data block.
  STARTED TASKS: Yes
  DONE: Not yet. The core logic works, but it still needs polishing for a complete and stable release.

8. Task Status Tracking

TASKS:

- Display the current status of each task (Pending, In Progress, Completed).
- Allow the user to manually update a task's progression status.
  Non-functional requirements:
- State management must be handled via Enums to avoid typo-based bugs during string comparisons.
- Status changes must trigger an immediate localized write to `users.json` via Gson to prevent data loss if the program is closed abruptly.
  STARTED TASKS: Yes
  DONE: Not yet. The core logic works, but it still needs polishing for a complete and stable release.

9. Overdue Task Alert

TASKS:

- Automatically check task deadlines against the current system date.
- Visually label past-due tasks as "Overdue" in the terminal output.
  Non-functional requirements:
- Deadline comparisons must rely strictly on the `java.time.LocalDate` API for accurate, timezone-independent date arithmetic.
  STARTED TASKS: Yes

DONE: Not yet. The core logic works, but it still needs polishing for a complete and stable release.

10. External Calendar Synchronization (Export & Sync)

TASKS:

- Allow the user to export schedules and task deadlines for use in external calendars.
  Non-functional requirements:
- The export function must generate standard formats (like `.csv` or basic `.ics` text structures) using `java.io.FileWriter`.
- The system must handle `IOException` gracefully if the local system denies write permissions in the working directory.
  STARTED TASKS: Yes
  DONE: Not yet. The core logic works, but it still needs polishing for a complete and stable release.

11. Focused Study Session Timer

TASKS:

- Implement interval-based focus cycles (Pomodoro-style).
- Coordinate the study timer with the Quick Study Break reminder module.
  Non-functional requirements:
- The timer state must remain stable in memory; if the user navigates to a different CLI menu, the countdown should continue accurately in the background.
  STARTED TASKS: Yes
  DONE: Not yet. The core logic works, but it still needs polishing for a complete and stable release.

12. Backup & Data Export

TASKS:

- Create a tool to allow students to manually back up their local data.
- Export all core data structures (schedules, tasks, budgets) to a secure file.
  Non-functional requirements:
- The backup process must safely duplicate `users.json` into a separate backup file (e.g., `backup.json`) without interrupting the main read/write stream to prevent file corruption.
  STARTED TASKS: Yes
  DONE: Not yet. The core logic works, but it still needs polishing for a complete and stable release.